Mixture of Experts Models

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

Mixture of Experts Models

Scaling capacity without scaling compute

Table of Contents

Motivation

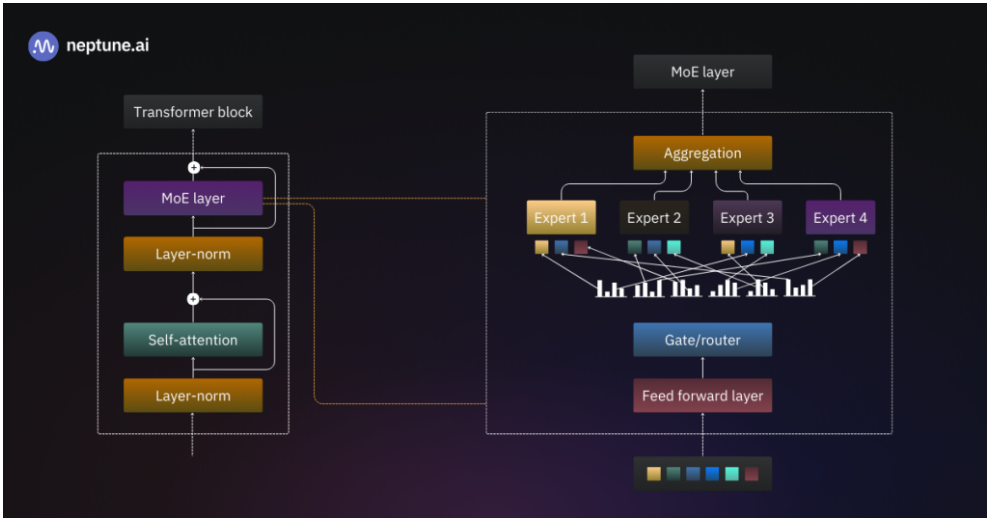
- **Why study Mixture of Experts?**
  - Scaling dense LLMs is costly: every parameter participates in every forward pass, so capacity and compute grow together.
  - **MoE models** break that link: massive total capacity with far lower computation per token, because only a few experts activate per input.
  - Many frontier systems are MoE models (e.g. DeepSeek-V3/R1 with 671B total but 37B active parameters; Mixtral; Switch Transformer).
- **Example:**
  - *A dense 671B model would be prohibitively expensive to serve; DeepSeek-R1 routes each token through a small subset of experts, giving 671B-scale capacity at roughly the inference cost of a 37B model.*
- Large Reasoning Models build directly on these architectures: several leading reasoning systems are trained and served as MoE models.

Learning Outcomes

By the end of this session, you should be able to:

- Explain the core idea of Mixture-of-Experts (MoE) architectures and why they decouple model capacity from per-token compute.
- Contrast sparse and dense models and reason about their trade-offs.
- Describe gating mechanisms, top-k routing, and noisy gating.
- Explain load balancing, the auxiliary loss, and expert-collapse failure modes.
- Identify how MoE layers integrate into Transformers (Switch Transformer, GShard, Mixtral, DeepSeek) and how expert parallelism distributes them.
- Assess the benefits and limitations of MoE models in practice.

What is a Mixture of Experts (MoE)?


neptune.ai
Transformer block
MoE layer
Layer-norm
Self-attention
Layer-norm
MoE layer
Aggregation
Expert 1
Expert 2
Expert 3
Expert 4
Gate/router
Feed forward layer

What is a Mixture of Experts (MoE)?

- **Definition**
  - It combines multiple specialized models, called experts.
  - Each expert focuses on a specific subset of data or task.
  - This specialization enables efficient and effective learning.
- **How does it work?**
  - MoE uses a gating mechanism.
  - The gate selects which expert(s) to use for a given input.
  - The model adapts its behavior dynamically based on the input.
  - It leverages the strengths of different experts.

What is a Mixture of Experts (MoE)? (cont.)

- **Why is it important?**
  - MoE models handle large-scale problems efficiently.
  - They distribute workload across multiple experts.
  - Combining expertise of different models improves performance on complex tasks.
- **Challenges**
  - Training MoE models is computationally expensive.
  - Multiple experts need to be trained.
  - Designing effective gating mechanisms is difficult.
  - Ensuring balanced expert utilization is challenging.

What is a Mixture of Experts (MoE)? (cont.)

Examples of Mixture of Experts Models

- **Switch Transformer**
  - Uses a gating mechanism to select one or a few experts per input.
  - Activates only a small subset of experts at a time.
  - Enables large-scale models without excessive computational costs.
  - *Example:*
    - Handles language modeling and translation.
    - Leverages specialized experts for different aspects of the task.

What is a Mixture of Experts (MoE)? (cont.)

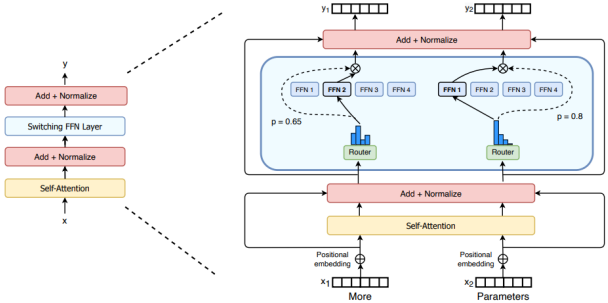


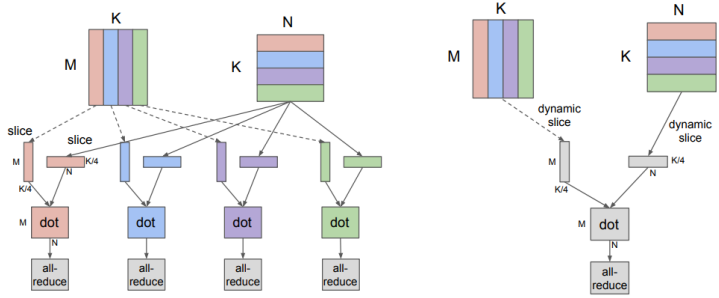
Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Source: Fedus et al. (2021)

What is a Mixture of Experts (MoE)? (cont.)

- **GShard**
  - Scales up the number of experts for very large datasets and models.
  - Distributes computation across multiple devices.
  - Enables efficient training of large models.
  - *Example:*
    - Used for image classification and NLP tasks.
    - Utilizes many experts to process different data parts.

What is a Mixture of Experts (MoE)? (cont.)



(a) MPMD Partition (b) SPMD Partition

Figure 2: Comparison between MPMD and our proposed SPMD partitioning of a Dot operator ($[M, K] \times [K, N] = [M, N]$) across 4 devices. In this example, both operands are partitioned along the contracting dimension K, where each device computes the local result and globally combines with an AllReduce. MPMD partitioning generates separate operators for each device, limiting its scalability, whereas SPMD partitioning generates one program to run on all devices. Note that the compilation time with our SPMD partitioning is not-dependent of the number of devices being used.

Source: Lepikhin et al. (2020)

What is a Mixture of Experts (MoE)? (cont.)

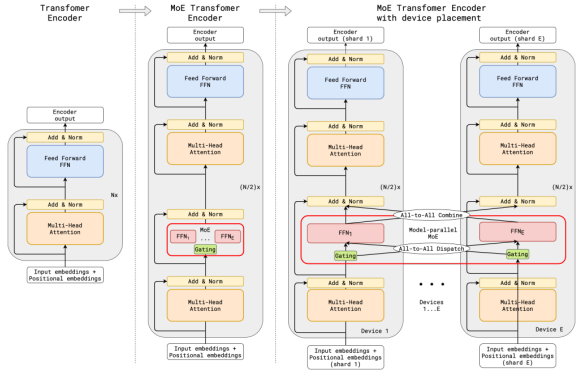


Figure 3: Illustration of scaling of Transformer Encoder with MoE Layers. The MoE layer replaces the every other Transformer feed-forward layer. Decoder modification is similar. (a) The encoder of a standard Transformer model is a stack of self-attention and feed forward layers interleaved with residual connections and layer normalization. (b) By replacing every other feed forward layer with a MoE layer, we get the model structure of the MoE Transformer Encoder. (c) When scaling to multiple devices, the MoE layer is sharded across devices, while all other layers are replicated.

Source: Lepikhin et al. (2020)

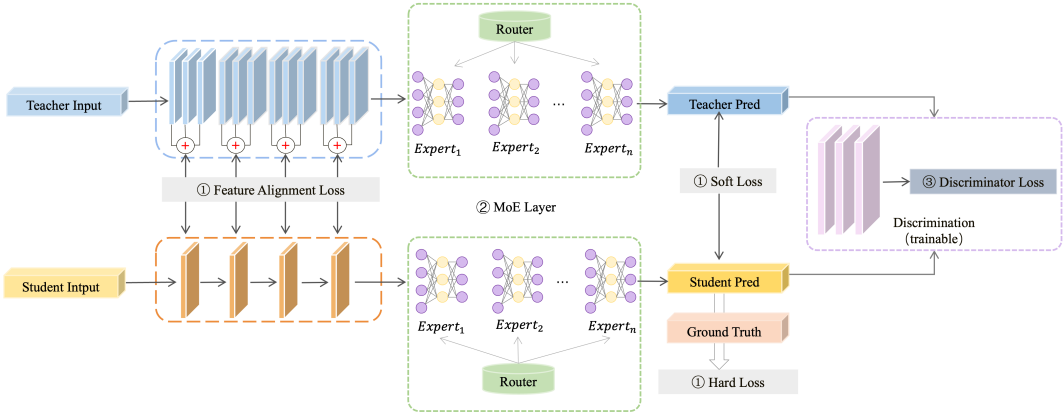
What is a Mixture of Experts (MoE)? (cont.)

- **MoE in BERT**
  - BERT can be extended with MoE layers.
  - MoE layers enhance performance on specific tasks.
  - Allows BERT to dynamically select experts based on input.
  - Improves ability to handle diverse tasks.
  - *Example:*
    - MoE-enhanced BERT performs better on sentiment analysis and question answering.
    - Uses specialized experts for different input types.
- **MicroBERT: Distilling MoE-Based Knowledge**
  - Distills knowledge from MoE-enhanced BERT into a smaller model.
  - Maintains high performance with reduced parameters.
  - Uses knowledge distillation techniques.

What is a Mixture of Experts (MoE)? (cont.)

- Targets resource-constrained environments.
- *Key Points:*
  - Teacher: MoE-BERT; Student: MicroBERT.
  - Transfers both general and expert-specific knowledge.
  - Achieves competitive accuracy on NLP benchmarks.
  - Reduces inference time and memory usage.
  - Suitable for deployment on edge devices.

What is a Mixture of Experts (MoE)? (cont.)

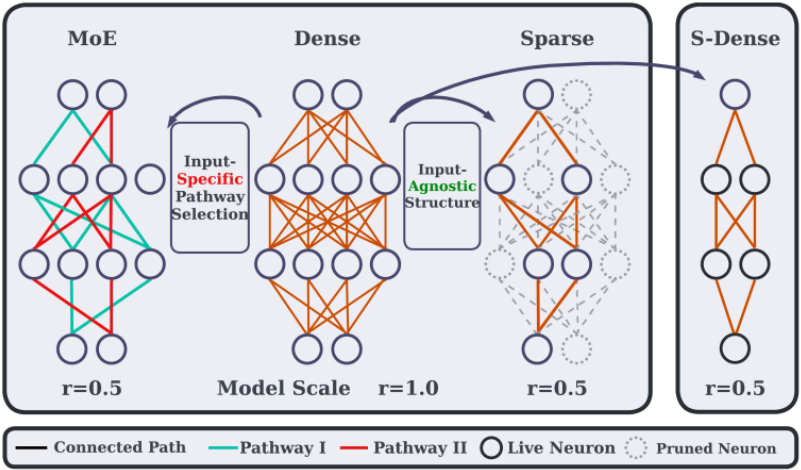


Source: Zheng et al. (2024)

What is a Mixture of Experts (MoE)? (cont.)

- **Other MoE Models**
  - MoE variants of these families exist: the Switch Transformer is built on T5, and GLaM matches GPT-3 quality using an MoE architecture.
  - MoE enables dynamic expert selection based on input.
  - Improves learning efficiency and effectiveness.

Sparse vs. Dense Models: **Trade-offs & Motivations**



Source: Zhang et al. (2023)

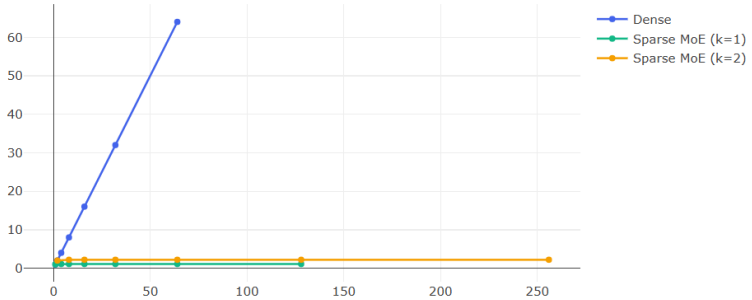
Sparse vs. Dense Models: Trade-offs & Motivations

- **Dense Models:**
  - Use a large number of parameters.
  - All parameters participate for every input.
  - Compute cost is predictable and uniform.
  - Training is straightforward: no routing or selection needed.
  - Require more computational resources.
  - **Examples:**
    - Standard Transformer models (e.g., BERT, GPT).
    - Fully-connected neural networks.

Sparse vs. Dense Models: Trade-offs & Motivations (cont.)

- **Sparse Mixture-of-Experts (MoE) Models:**
  - Only a subset (top-k) of experts are activated per input.
  - Enables scaling to billions of parameters without increasing per-token compute.
  - Requires a routing mechanism to select experts.
  - Focus on specific features or patterns in the data.
  - Efficient for large datasets with sparse representations.
  - **Examples:**
    - Switch Transformer (Google).
    - GLaM (Google).
    - M6-T (Alibaba).

Sparse vs. Dense Models: Trade-offs & Motivations (cont.)



Comparison showing how FLOPs per token scale with total model parameters. Dense models exhibit coupled growth, while sparse MoE models allow parameter scaling with nearly constant FLOPs per token (assuming fixed k). Note the slight increase for MoE accounts for gating overhead and assumes experts scale parameters. Axes represent relative scale.

Source: Contrasting Dense vs. Sparse Activation

Sparse vs. Dense Models: Trade-offs & Motivations (cont.)

- **Trade-offs:**
  - **Dense:** Simpler, stable, but limited by compute and memory.
  - **Sparse:** Higher capacity, efficient compute, but adds routing overhead.
  - Sparse models may face instability and load balancing issues.
  - Choice depends on scale, task complexity, and infrastructure.

Gating Mechanisms – **How Experts Are Selected**

Gating Mechanisms – How Experts Are Selected

- **Gating Mechanism:**
  - A crucial component in Mixture of Experts (MoE) models determines which expert(s) to activate for a given input.
  - Uses a learned function to compute a score for each expert.
  - The scores are used to select the top-k experts.
  - (A small neural network that selects the top-k experts to use for a given input.)

Gating Mechanisms – How Experts Are Selected (cont.)

- **How it works:**
  - Input is passed through a gating network.
  - The gating network outputs a probability distribution over experts.
  - Only the experts with the highest probabilities are activated.
  - This allows the model to focus on relevant experts for each input.

Gating Mechanisms – How Experts Are Selected (cont.)

- **MoE Formulation**
  - Input token: x
  - Gating network G calculates expert distribution.
- **Gating Network Calculation**
  - Linear transformation: $logits = W_g x$
  - Softmax activation: $P = \text{softmax}(logits)$
  - W_g: trainable weight matrix
  - P: probability vector for each expert

Gating Mechanisms – How Experts Are Selected (cont.)

- **Dense MoE Output**
  - Output $y = \sum_{i=1}^{N} P_i E_i(x)$
  - Combines all experts with weights P_i.
- **Sparse MoE Goal**
  - Focus on computational efficiency.
  - Only activate a small subset of experts per token.
  - Requires a mechanism to select experts based on P.

Gating Mechanisms – How Experts Are Selected (cont.)

- **Prevalent Strategy**
  - For sparse MoEs, selects a subset of experts.
  - Gating network chooses k experts with highest probabilities.
  - Typically, k is small (e.g., 1 or 2).

Gating Mechanisms – How Experts Are Selected (cont.)

- **Process Overview**
  - **Compute Scores:** Calculate gating logits or probabilities (P).
  - **Select Top-k:** Identify indices $I = \{i_1, i_2, \ldots, i_k\}$ for top k values in P.
  - **Re-normalize Scores (Optional):** take P_{sel}, the sub-vector of P for selected indices I, and re-normalize:

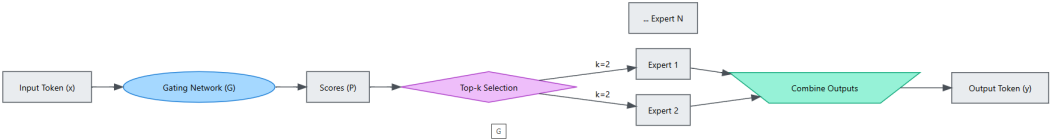
$$P'_j = \frac{\exp\left(\text{logits}_{i_j}\right)}{\sum_{l=1}^{k} \exp\left(\text{logits}_{i_l}\right)} \quad \text{for } j = 1 \ldots k$$

  - **Compute Weighted Output:** $y = \sum_{j=1}^{k} P'_j E_{i_j}(x)$, using the (re-normalized) probabilities as weights.

Gating Mechanisms – How Experts Are Selected (cont.)

- **Choice of k**
  - $k = 1$: Token routed to a single expert; maximizes sparsity but limits knowledge combination.
  - $k = 2$: Token uses two experts; balances sparsity and representational capacity at double the computational cost of $k = 1$.
  - k is a key hyperparameter affecting performance and cost.

Gating Mechanisms – How Experts Are Selected (cont.)



Source: Designing Effective Gating Networks

Gating Mechanisms – How Experts Are Selected (cont.)

- **Load-Balanced + Entropy-Regularized Gating**
  - Ensures efficient and equal expert utilization.
  - Prevents overloaded or underutilized experts.
  - **Load-Balanced:**
    - Penalty in loss encourages even token distribution.
    - Avoids "expert collapse."
  - **Entropy-Regularized:**
    - Promotes diverse expert selection.
    - Nudges model to use full range of experts.

Gating Mechanisms – How Experts Are Selected (cont.)

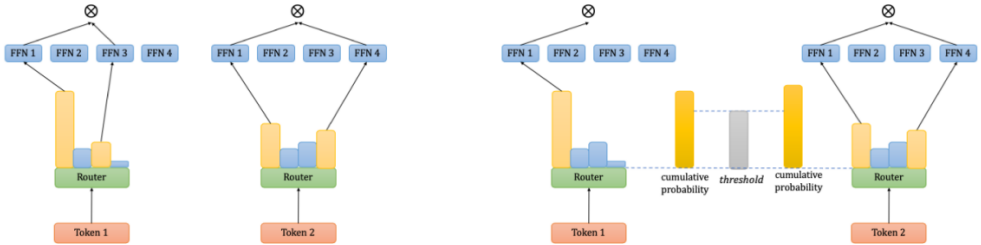
- **Examples in Practice**
  - **Example 1: Switch Transformer Gating**
    - Primarily uses **Top-1 routing**.
    - Incorporates **load balancing penalty**.
    - Ensures experts are actively learning and distributed.
  - **Example 2: MoE-LLaMA**
    - Employs **Top-2 gating**.
    - **Avoids token dropout**.
    - Aims for smoother and consistent performance.

Routing in MoE

Routing in MoE

- **Key Concepts:**
  - Assign tokens to experts per layer.
  - Routing can be token-level or batch-level.
  - Penalize overused experts (using load balancing loss).
  - Efficient hardware utilization is key.

Routing in MoE (cont.)



(a) Top-K routing

(b) Top-P routing

Figure 1: Comparison between Top-K routing mechanism and Top-P routing mechanism. (a) Each token selects fixed K=2 experts with Top-K routing probabilities. (b) In Top-P routing mechanism, each token selects experts with higher routing probabilities until the cumulative probability exceeds threshold.

Source: Huang et al. (2024)

Routing in MoE (cont.)

- **Example 1: DeepSpeed-MoE**
  - Optimizes distributed routing and communication.
  - Achieves linear scaling to hundreds of experts.
  - Focuses on efficient data movement across devices.

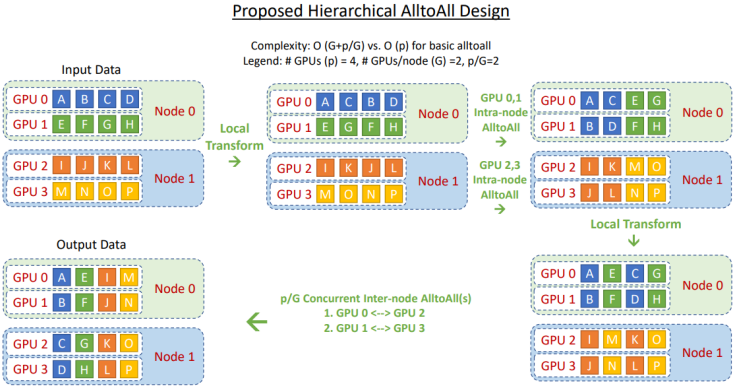


Figure 8: Illustration of the proposed hierarchical all-to-all design.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

Routing in MoE (cont.)

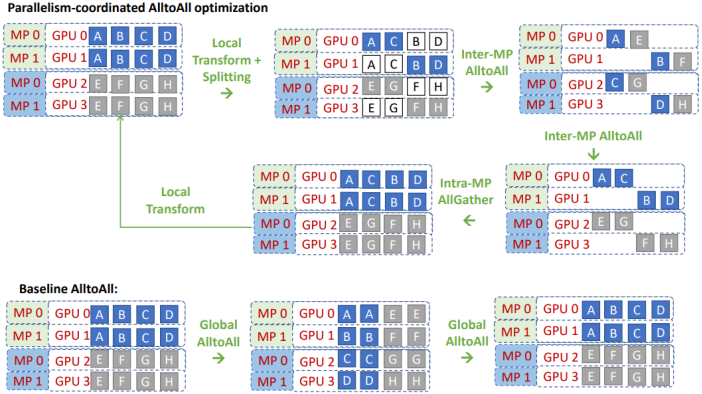


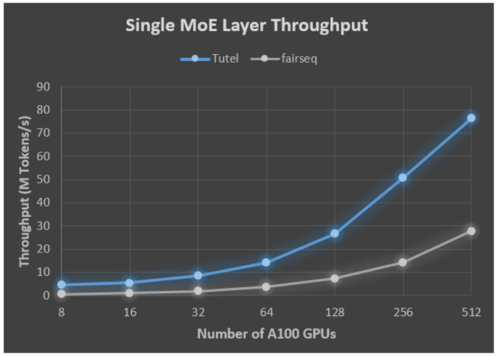
Figure 9: Illustration of the parallelism coordinated communication.

Source: DeepSpeed-MoE, Rajbhandari et al. (2022)

Routing in MoE (cont.)

- **Example 2: Tutel by Microsoft**
  - Uses custom **CUDA kernels**.
  - For highly efficient token routing in MoE.
  - Reduces overhead and speeds up computation on GPUs.

Routing in MoE (cont.)



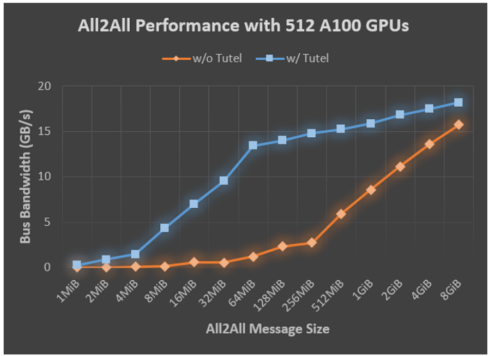


Figure 1 (left): Compared to fairseq, for a single MoE layer, Tutel achieves an 8.49x speedup on an NDm A100 v4 node with 8 GPUs and a 2.75x speedup on 64 NDm A100 v4 nodes with 512 A100 GPUs. The detailed setting is as follows: batch_size = 32, sequence_length = 1,024, Top_K = 2, model_dim = 2,048, and hidden_size = 2,048. Figure 2 (right): The all-to-all bandwidth for different message sizes with 64 NDm A100 v4 nodes (512 A100 GPUs) before and after applying Tutel. Tutel achieves a 2.56x to 5.93x all-to-all speedup with 512 A100 GPUs for message sizes hundreds of MiB large.

Source: Tutel (2021)

MoE at Scale: Load Balancing, Switch Transformers, and Parallelism

Notable MoE Papers

- Switch Transformer (Fedus et al., 2021): Efficient routing and scaling.
- GShard (Lepikhin et al., 2020): Large-scale MoE for multilingual translation.
- M6, GLaM, and recent GPT-MoE variants: Further advances in scaling and performance.

Load Balancing in MoEs

Problem: Without intervention, the gating network tends to route most tokens to a few experts, leading to inefficient training and poor expert utilization.

- **Expert Overload:** Popular experts receive more tokens, train faster, and become even more favored.
- **Underutilization:** Other experts receive few or no tokens, limiting their contribution.

Solution: Introduce an auxiliary loss to encourage balanced token distribution across experts.

- **Auxiliary Loss:** Penalizes uneven assignment of tokens, pushing the gating network to distribute tokens more uniformly.
- **Expert Capacity:** Each expert is assigned a maximum number of tokens it can process (capacity). Excess tokens are dropped or rerouted.
- **Implementation:** In transformer-based MoEs, this is often controlled via an `aux_loss` parameter.

Load Balancing in MoEs (cont.)

Auxiliary Loss Example:

$$L_{\text{aux}} = n \cdot \sum_{i=1}^{n} f_i \log f_i$$

where f_i is the fraction of tokens assigned to expert i, and n is the number of experts.

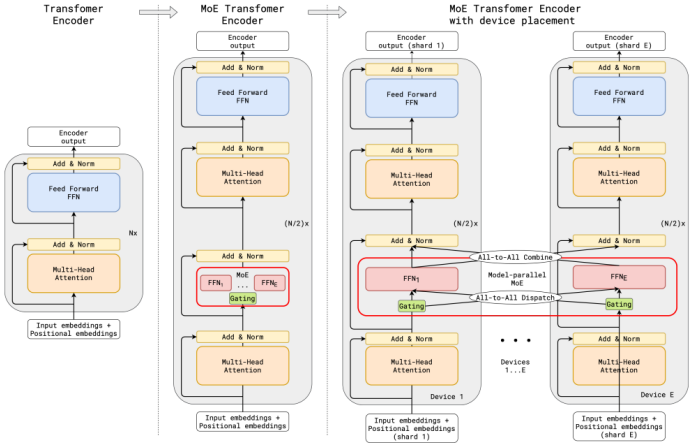
Benefits:

- Ensures all experts are trained and utilized.
- Improves model efficiency and generalization.
- Prevents bottlenecks and overload in popular experts.

Transformers and Scaling: Transformers demonstrate that increasing the number of parameters improves performance. Google's GShard project explored scaling transformers beyond 600 billion parameters using MoE layers.

- **GShard Approach:** Replaces every other feed-forward (FFN) layer with an MoE layer using top-2 gating in both encoder and decoder.
- **Distributed Training:** MoE layers are shared across devices, while other layers are replicated, enabling efficient large-scale computation.

MoEs and Transformers (cont.)



MoE Transformer Encoder from the GShard Paper

Load Balancing and Efficiency:

- **Random Routing:** In top-2 gating, the top expert is always selected, while the second expert is chosen probabilistically based on its gating weight.
- **Expert Capacity:** Each expert has a maximum token capacity. Overflow tokens are either sent via residual connections or dropped, ensuring static tensor shapes and balanced computation.

Why Expert Capacity?

- Tensor shapes must be fixed at compile time, but token distribution to experts is dynamic. Capacity factors ensure compatibility and efficiency.

Inference Efficiency:

- During inference, only a subset of experts is activated per input.
- Shared computations (e.g., self-attention) are applied to all tokens, so the effective compute is much lower than the total parameter count.
- Example: Mixtral 8x7B has about 47B total parameters but activates only about 13B per token with top-2 gating, so its inference compute is close to that of a 13B dense model.

Switch Transformers: Address training and fine-tuning instabilities in MoEs by simplifying routing and improving efficiency. The authors released a 1.6T parameter MoE model with 2048 experts on Hugging Face, achieving a 4x pre-train speed-up over T5-XXL.

- **Switch Transformer Layer:** Replaces FFN layers with MoE layers, but routes each token to a single expert (single-expert strategy), reducing router computation and communication costs while preserving quality.
- **Expert Capacity:** Defined as

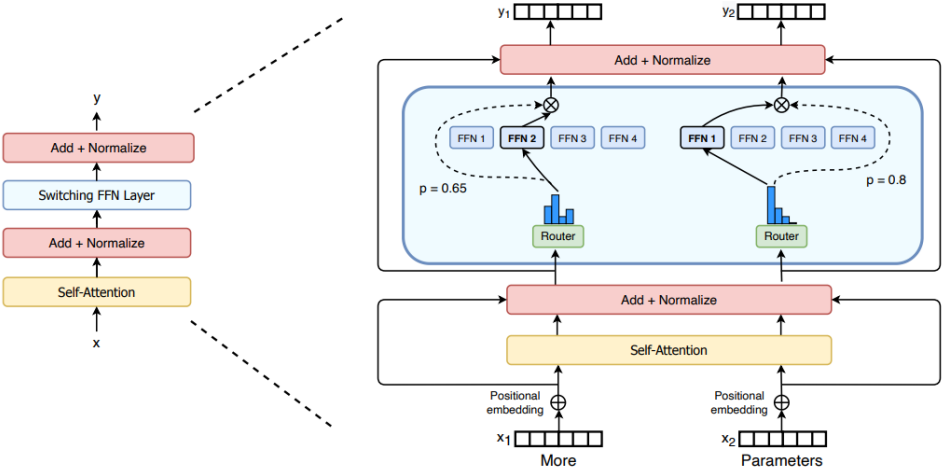
$$\text{Expert Capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}}\right) \times \text{capacity factor}$$

- Low capacity factors (1–1.25) are effective, balancing efficiency and communication overhead.
- **Load Balancing Loss:** Simplified auxiliary loss encourages uniform routing and is weighted by a hyperparameter.

Switch Transformers (cont.)

- **Selective Precision:** Experts are trained in bfloat16, while routing uses full precision to avoid instability. This reduces communication and computation costs without degrading quality.

Switch Transformers (cont.)



Switch Transformers (cont.)

Architecture:

- Encoder-decoder setup, similar to T5 but with MoE layers.
- Fine-tuning and summarization tasks are supported.

GLaM and Scaling:

- GLaM explores scaling MoEs further, matching GPT-3 quality with 1/3 the energy.
- Uses decoder-only models, top-2 routing, and larger capacity factors.
- Capacity factor can be adjusted during training and inference to control compute and efficiency.

Parallelism in MoE

Parallelism Overview:

- **Data Parallelism:** Replicates model weights across all cores; data is partitioned so each core processes a different batch.
- **Model Parallelism:** Splits the model across cores; each core holds a part of the model, and data is replicated.
- **Model and Data Parallelism:** Both model and data are partitioned; different cores process different batches and hold different model segments.
- **Expert Parallelism:** Experts are distributed across workers. Each worker processes a different batch, and MoE layers route tokens to the appropriate expert on each worker.

Expert Parallelism Details:

- For non-MoE layers, expert parallelism acts like data parallelism.
- For MoE layers, tokens are dynamically routed to the worker hosting the selected expert.

Parallelism in MoE (cont.)

- Enables scaling to large numbers of experts and efficient distributed training.

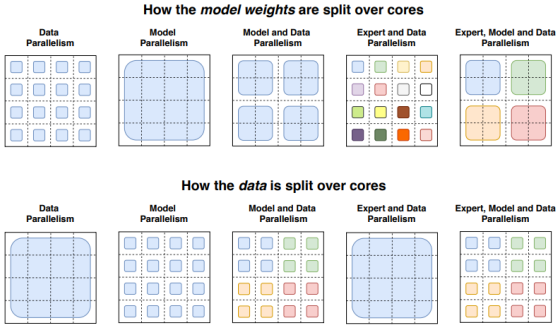


Illustration of model, expert, and data parallelism (adapted from Switch Transformers)

Benefits of MoE

Benefits of MoE

- **Scalability:**
  - Can scale to billions of parameters.
  - Only a small subset of experts is activated for each input.
  - Allows for larger models without proportional increases in compute.

Benefits of MoE (cont.)

- **Efficient Computation:**
  - Activates only 10–20% of total parameters per token.
  - Reduces computational cost per token.
  - Sparse activation means fewer parameters are active at any time.
  - Enables training on larger datasets with limited resources.
  - Leads to lower latency and inference cost.
  - Allows for models with vastly more parameters.

Benefits of MoE (cont.)

- **Flexibility:**
  - Different experts can specialize in different tasks or data distributions.
  - Allows for more nuanced model behavior and adaptability.
  - Can be tailored to specific applications or domains.

Benefits of MoE (cont.)

- **Specialization:**
  - Experts can focus on specific domains (e.g., math, code, biology, logic).
  - Improves accuracy and performance in specialized tasks.
- **Example 1: MoE with Domain Experts**
  - A math expert becomes active only on arithmetic inputs.
  - Improves accuracy specifically for math problems.
  - Outperforms generalist models on these tasks.
- **Example 2: MORE (Mixture of Reasoning Experts)**
  - Explicitly uses experts for types of reasoning (e.g., multihop, factual, analogical).
  - Boosts performance and provides greater interpretability.
  - Allows understanding which expert handles which reasoning step.

Benefits of MoE (cont.)

- **Performance:**
  - Often achieves state-of-the-art results on various benchmarks.
  - Combines the strengths of multiple models without the overhead of training them separately.
  - Can outperform dense models with fewer active parameters.

Limitations of MoE Models

Limitations of MoE Models

- **Training instability:** Some experts may be underused, leading to "expert collapse."
- This is often addressed with load balancing.
- **Hard to debug:** Expert specialization may not always align with human-interpretable domains.
- Makes understanding model behavior challenging.
- **Distributed training cost:** Load balancing across GPUs adds significant engineering overhead.
- Requires complex distributed systems.

Limitations of MoE Models (cont.)

- **Example 1: GShard training instability**
  - Some experts never activate after several epochs of training.
  - Requires manual intervention or advanced balancing techniques.
- **Example 2: Switch Transformer**
  - Needed custom gradient balancing.
  - To prevent expert overload or dropout during training.

References & Further Reading

References & Further Reading

- DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948.
- Financial Times. (2025). *Transcript: The story behind DeepSeek's breakthrough*. ft.com.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., & Dean, J. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. arXiv:1701.06538.
- Shen, S., Hou, L., Zhou, Y., Du, N., Longpre, S., Wei, J., Chung, H. W., Zoph, B., Fedus, W., Chen, X., et al. (2023). *Mixture-of-Experts Meets Instruction Tuning: A Winning Combination for Large Language Models*. arXiv:2305.14705.
- NVIDIA Developer Blog. (2023). *Applying Mixture of Experts in LLM Architectures*. developer.nvidia.com.
- Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., & Huang, J. (2024). *A Survey on Mixture of Experts in Large Language Models*. arXiv:2407.06204.

References & Further Reading (cont.)

- DataCamp. (2024). *What Is Mixture of Experts (MoE)? How It Works, Use Cases & More*. datacamp.com.
- Si, C., et al. (2023). *Getting MoRE out of Mixture of Language Model Reasoning Experts*. In *Findings of EMNLP 2023*. ACL Anthology.